

Databases

Lecture 4 - Relationships

Emily Lucia Antosch

HAW Hamburg

04.04.2026

Contents

1. Introduction	7
2. Relationships	11
Basics	12
Basics	13
Entity Type	14
Company Example	16
Abstraction	17
Role Names	18
Recursive Relationship Types	19
Recursive Relationship Types	20
Constraints	22
Company Example	23

Constraints	24
Constraints	25
Constraints	26
Constraints	28
Company Example	29
Attributes	30
Company Example	31
How to define them?	32
Constraints	33
Referential Integrity Constraint	34
Referential Integrity Constraints	35
Company Example	36
Company Example	37

Notation of Foreign Keys	38
Notation of Foreign Keys	40
Notation of Foreign Keys	41
Mapping of ERM	42
Mapping of Binary 1:1	43
Mapping of Binary 1:1	44
Mapping of Binary 1:1	45
Mapping of Binary 1:1	46
Mapping of Binary 1:1	47
Mapping of Binary 1:1	48
Mapping of Binary 1:N	49
Mapping of Binary 1:N	50
Mapping of Binary 1:N - Total Participation	51

Mapping of Binary M:N	52
Mapping of Binary M:N	53
Mapping of Binary M:N	54
Example Company	55
Example Company	56
Assignment Office - Convert ERD to RM	57
Mapping of ERM to RM	58
Create Table - Constraints	59
Create Table - Constraints Syntax	60
Create Table - Constraints Syntax	61
Create Table - Constraints Syntax	62
Create Table - Constraints	63
Create Table - Constraints	64

Create Table - Constraints	65
ALTER TABLE	66
ALTER TABLE - Column	67
ALTER TABLE - Column	68
ALTER TABLE - Column	69
ALTER TABLE - Constraints	70
ALTER TABLE - Constraints	71
ALTER TABLE - Rename	72
Homework	73
3. License Notice	74

1. Introduction

1.1 Where are we right now?

- Last time, we looked at how to logically and conceptually design databases.
- You also learned about how to use simple SQL language to map a conceptually designed database to an actual one.
- Today, we'll look at
 - ▶ what relationships are in terms of databases,
 - ▶ how we can use constraints to enforce our relationships and,
 - ▶ how we can implement that in SQL.

1.1 Where are we right now?

1. Introduction
2. Basics
3. SQL
4. Entity-Relationship-Model
5. **Relationships**
6. Constraints
7. More SQL
8. Subqueries & Views
9. Transactions
10. Database Applications
11. Integrity, Trigger & Security

1.2 What is the goal of this chapter?

- At the end of this lesson, you should be able to
 - ▶ define relationships and identify which type of relationship that is and,
 - ▶ decide how to implement that relationship in SQL.

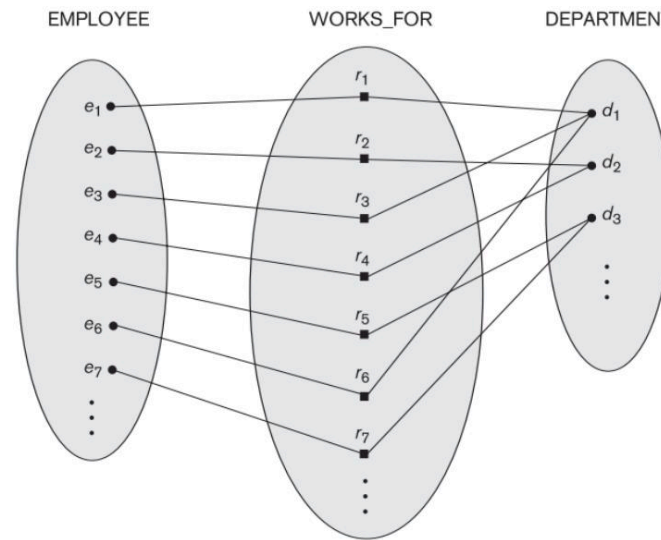
2. Relationships

Basics

- Describe relationships between entity types characterized by a verb
- Often 2 naming possibilities:
 - ▶ teaches vs. is taught by
 - ▶ Relationship has always two (or more) directions
- May have attributes
- Number of participating entity types (degree):
 1. Unary relationship type (e.g., Employee supervises another employee)
 2. Binary relationship type (e.g., Employee works for one department)
 3. Ternary relationship type (e.g., Lecturer recommends books for one specific course)
 4. Higher degrees...

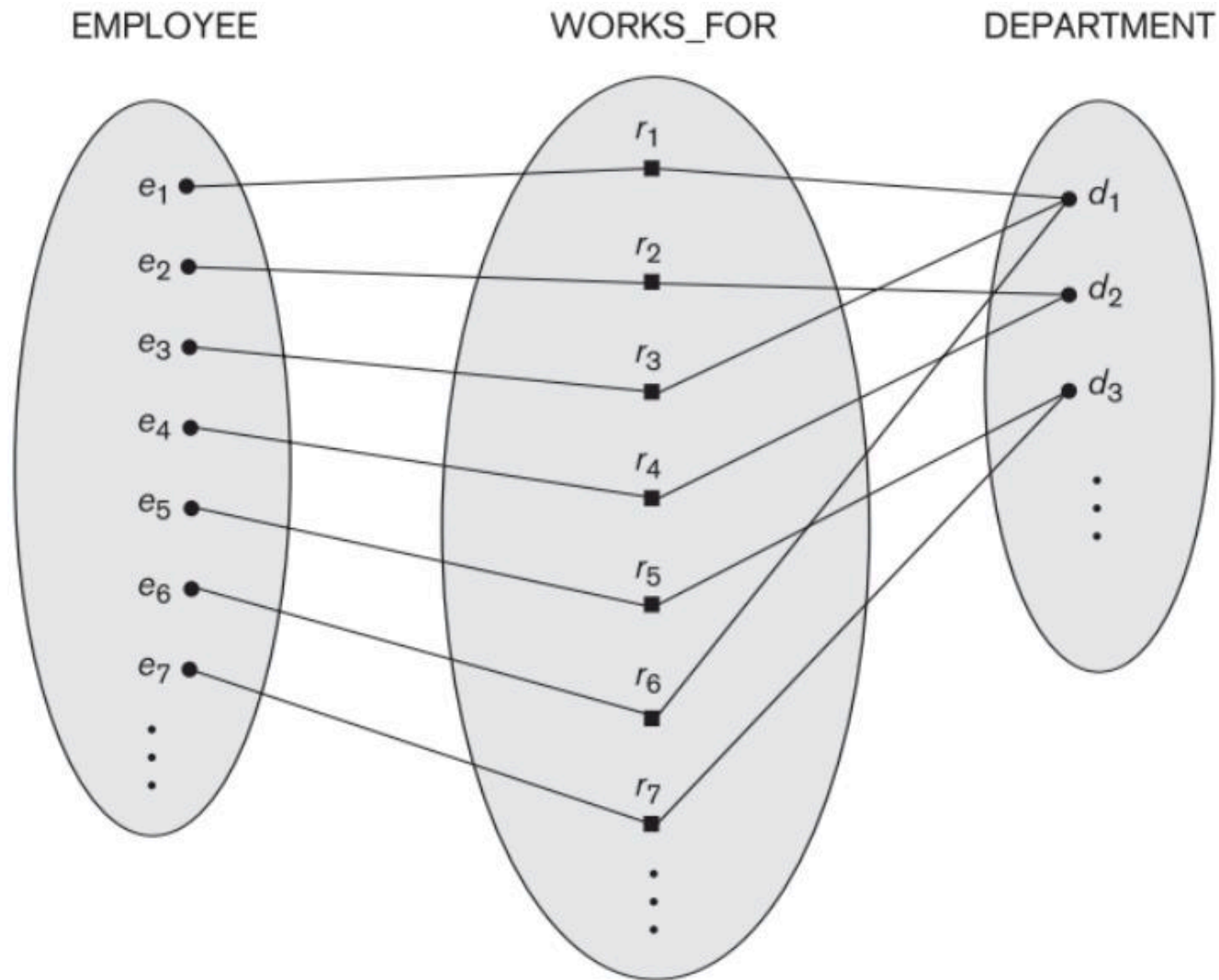
Basics

- Each relationship instance in R is an association of entities, where the association includes exactly one entity from each participating entity type
- In an ERM, relationship types are displayed as diamond-shaped boxes, which are connected by straight lines to the entity types

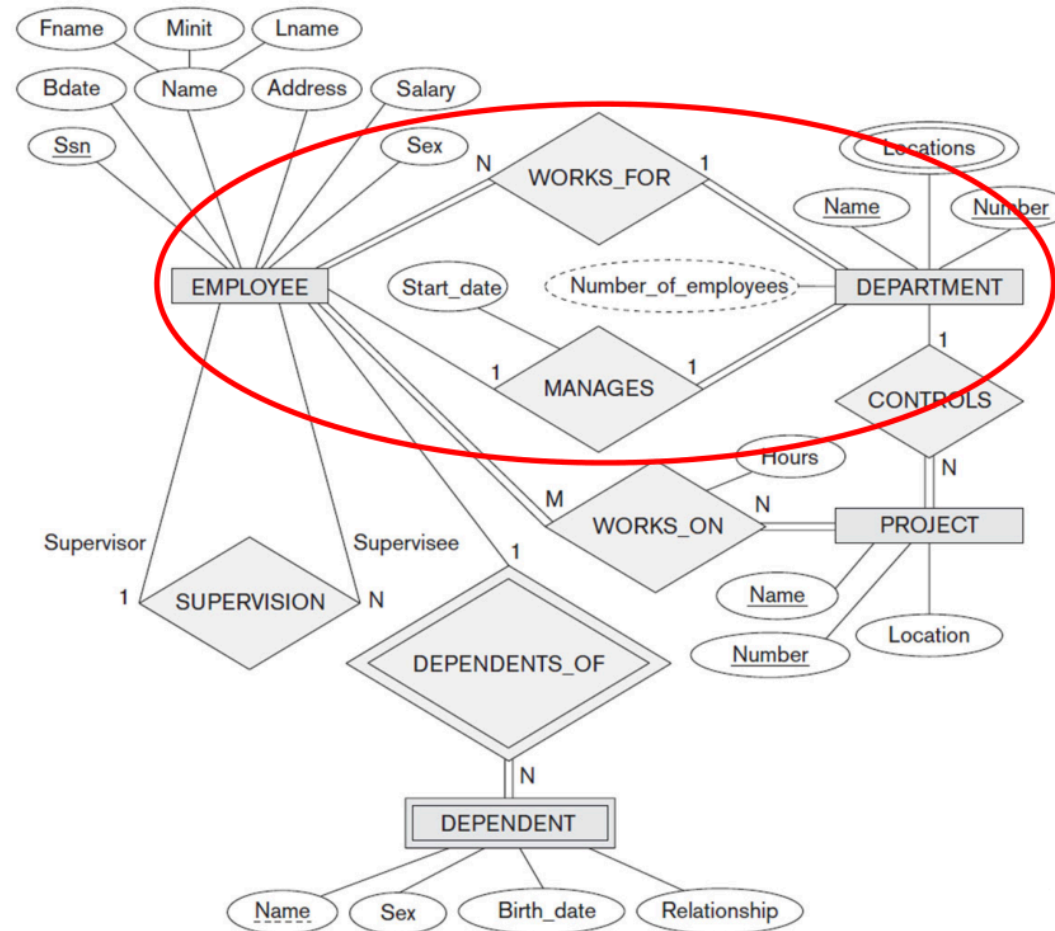


Entity Type

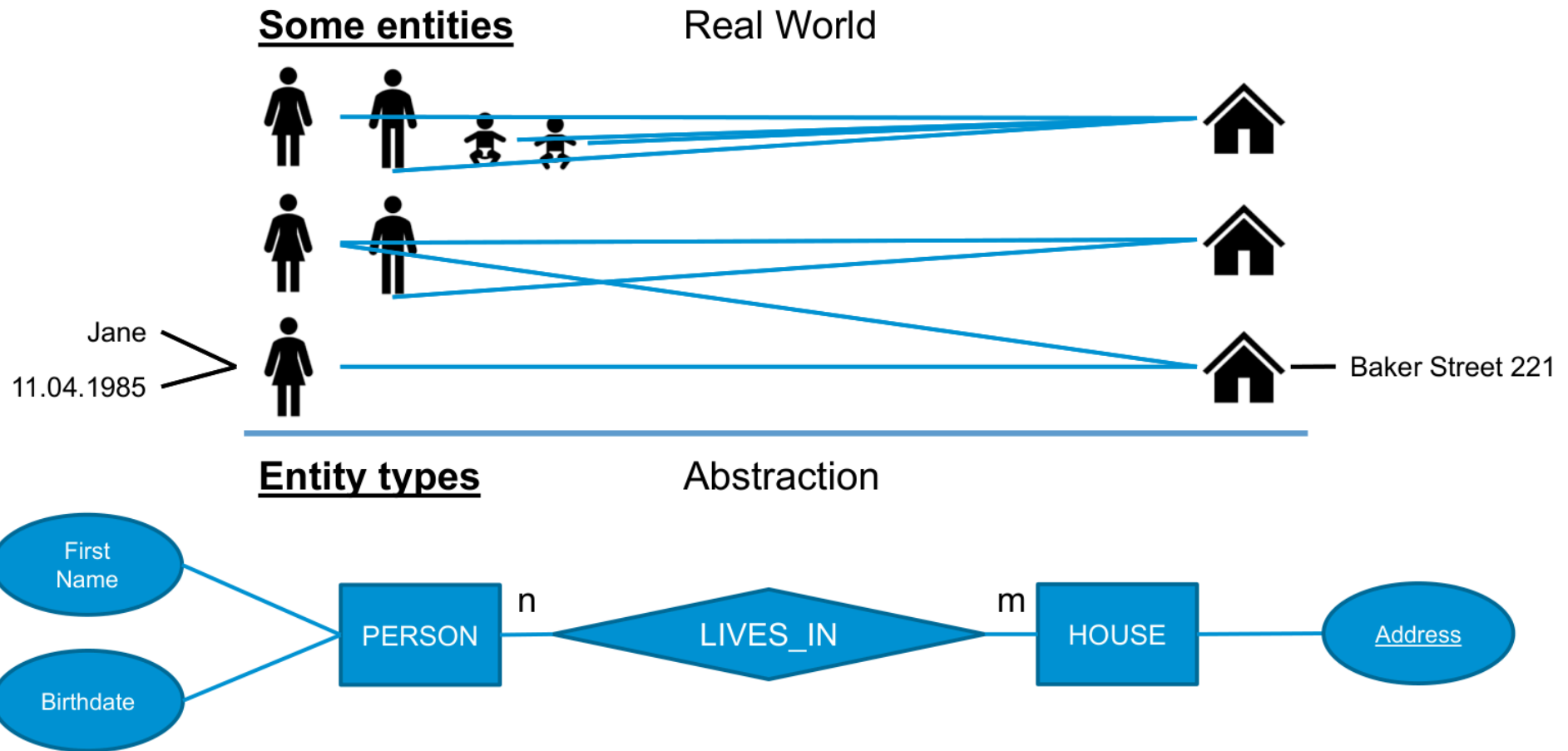
- Represented as rectangle in ERM
- Singular noun
- Attribute Type
 - ▶ Represented as ovals
 - ▶ Noun
- Relationship Type
 - ▶ Represented as diamond in ERM
 - ▶ Always between entity types
 - ▶ Verb
 - ▶ Has cardinalities



Company Example



Abstraction



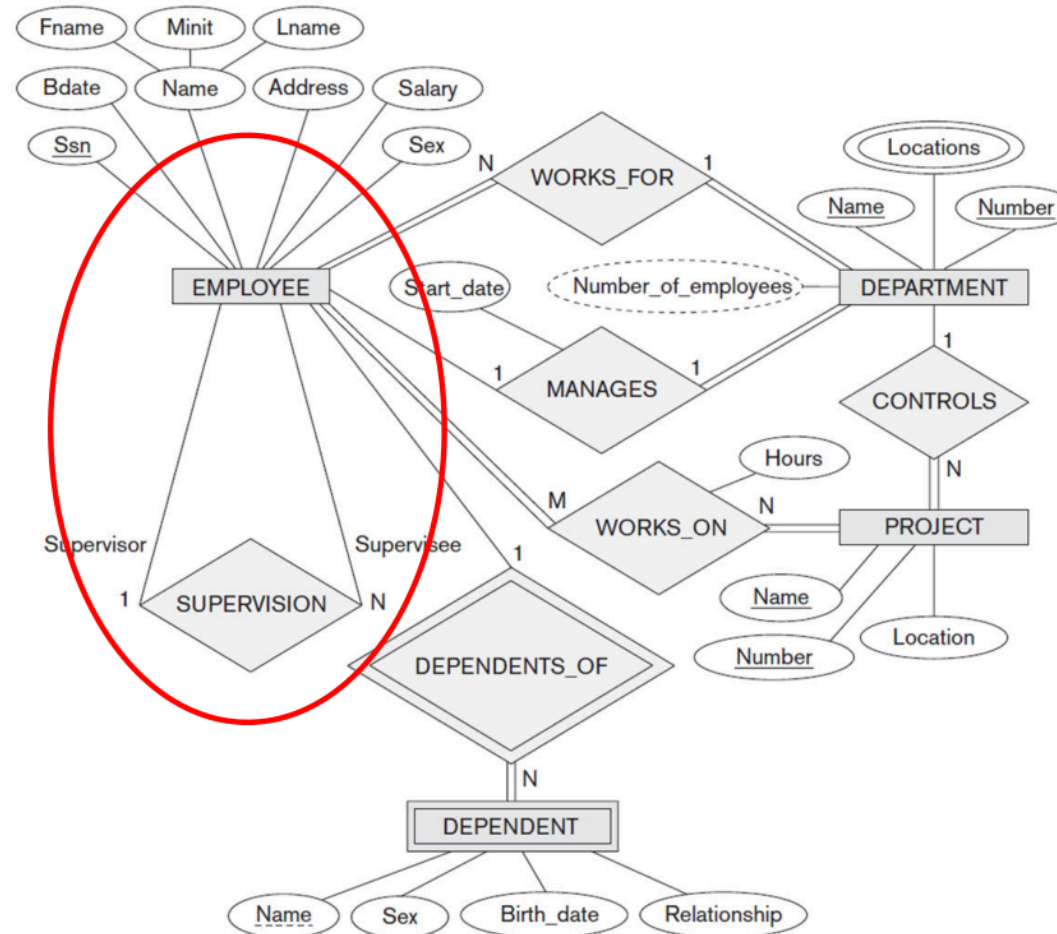
Role Names

- The **role name** signifies the role that a participating entity from the entity type plays in each relationship instance, and helps to explain what the relationship means

Recursive Relationship Types

- In some cases, the same entity type participates more than once in a relationship type in different roles
- In such cases the **role name** becomes essential for distinguishing the meaning of the role that each participating entity plays
- Such relationship types are called **recursive relationship types**

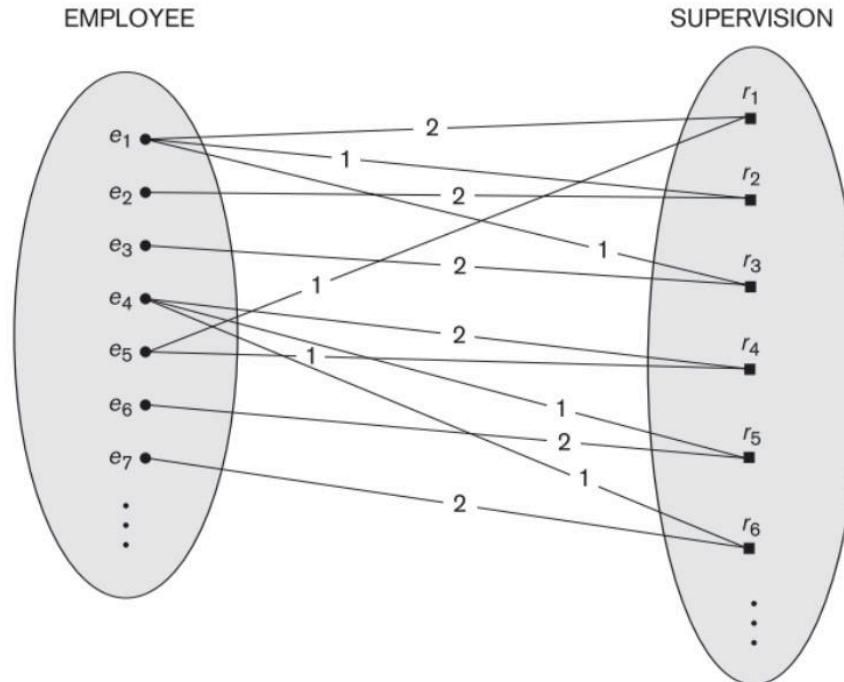
Recursive Relationship Types





Example: Employee in 2 roles

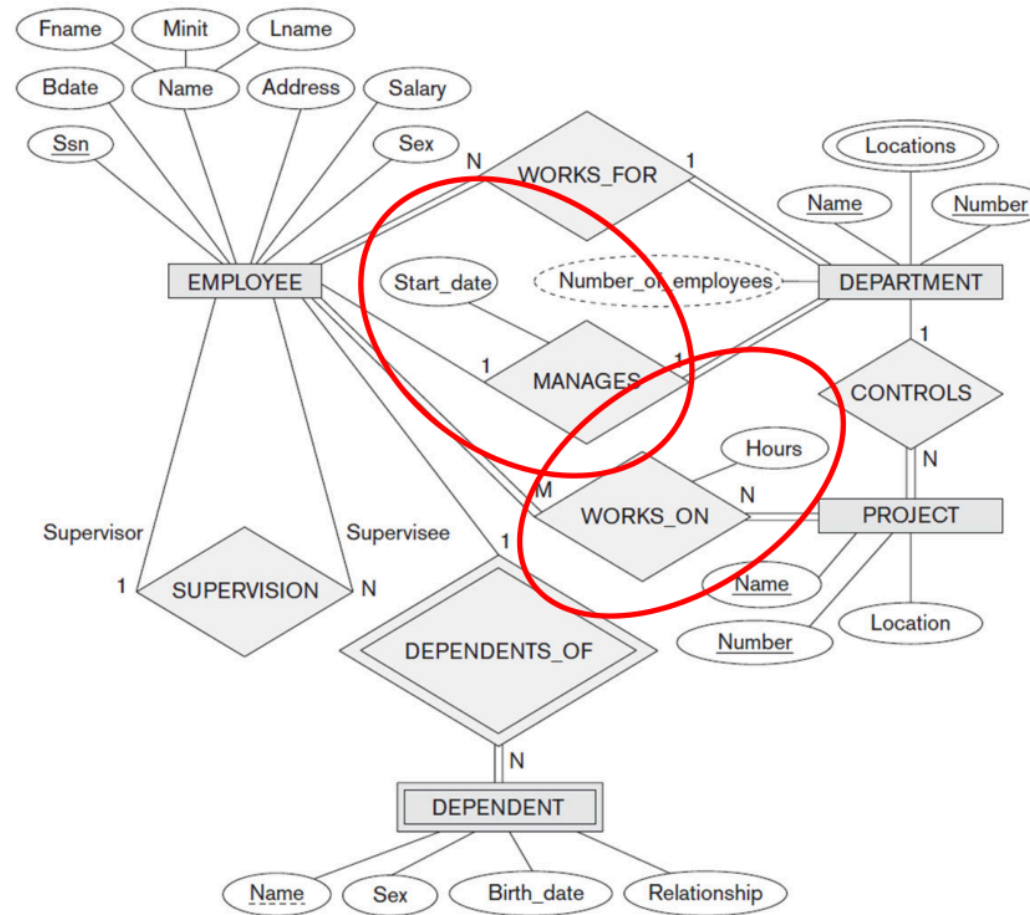
- Supervisor (boss) role name 1
- Supervisee (subordinate) role name 2



Constraints

- Cardinality
- Specifies the maximum number of relationship instances that an entity can participate in
- Cardinality ratios
 - ▶ 1:1
 - ▶ 1:N
 - ▶ M:N
- Cardinality ratios for binary relationships are represented on ER diagrams by displaying 1, M, and N on the diamonds
- Notice that in this notation, we can either specify no maximum (N) or a maximum of one (1) on participation

Company Example



Constraints

- Cardinality
- (min,max) Notation
- Example
 - ▶ A car has at least 3 and at most 5 wheels Every wheel is associated to exactly one car



- Attention: In UML, (min,max) is placed on the opposite sites!
- Problem: General case cannot be easily implemented in RDBMS

Constraints

- Other notations (e.g., in tools like draw.io)

■ 1:1



■ 1:N



■ M:N



Constraints

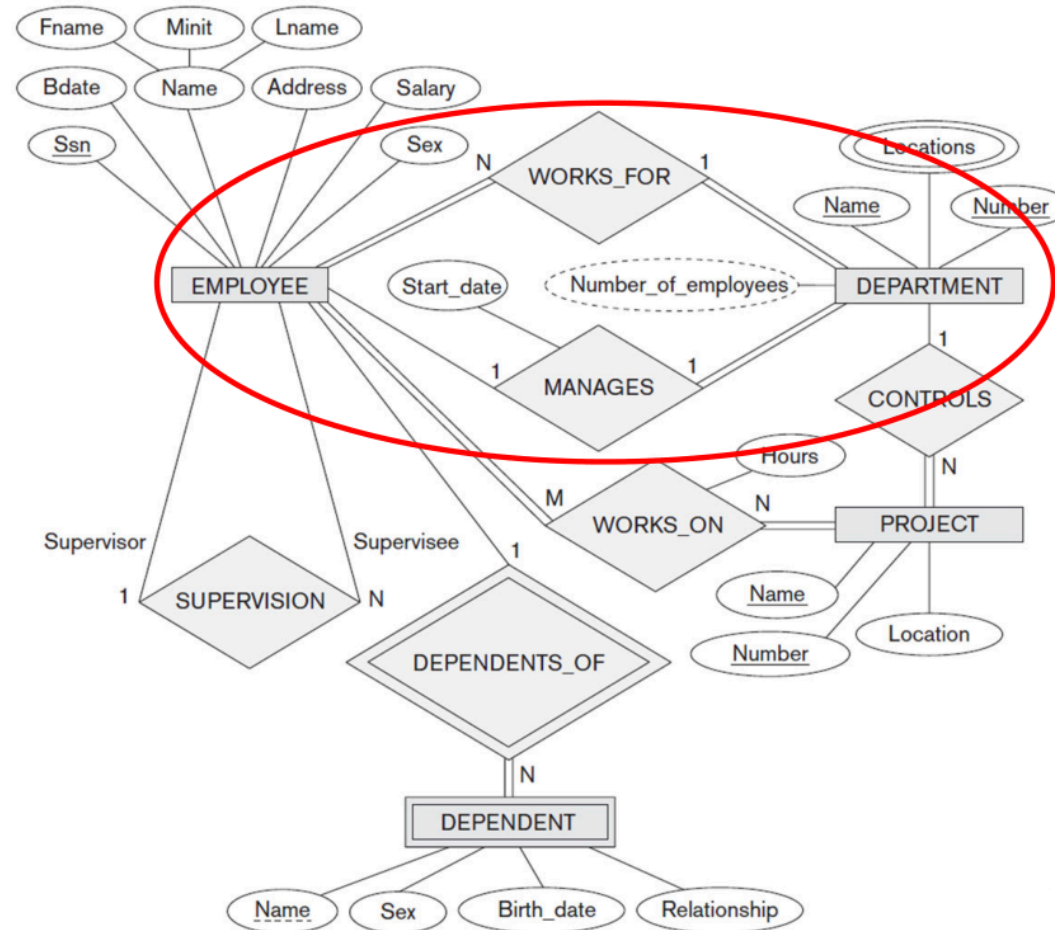
- Participation
- Specifies whether the existence of an entity depends on its being related to another entity via the relationship type
- Also called **minimum cardinality constraint**
- Two types
- Total: every entity in the total set of all entities of an entity type A must be related to an entity of entity type B via a relationship
 - ▶ Total participation is also called **existence dependency**
 - Is displayed as a double line connecting the participating entity type to the relationship
- Partial: some or part of the entities of an entity type A are related to some entities of an entity type B via a relationship

- ▶ Is displayed by a single line connecting the participating entity type to the relationship

Constraints

- Cardinality: specifies the maximum number of relationship instances that an entity can participate in
- Participation: specifies if the existence of an entity depends on its being related to another entity via the relationship type
 - ▶ **minimum cardinality constraint**

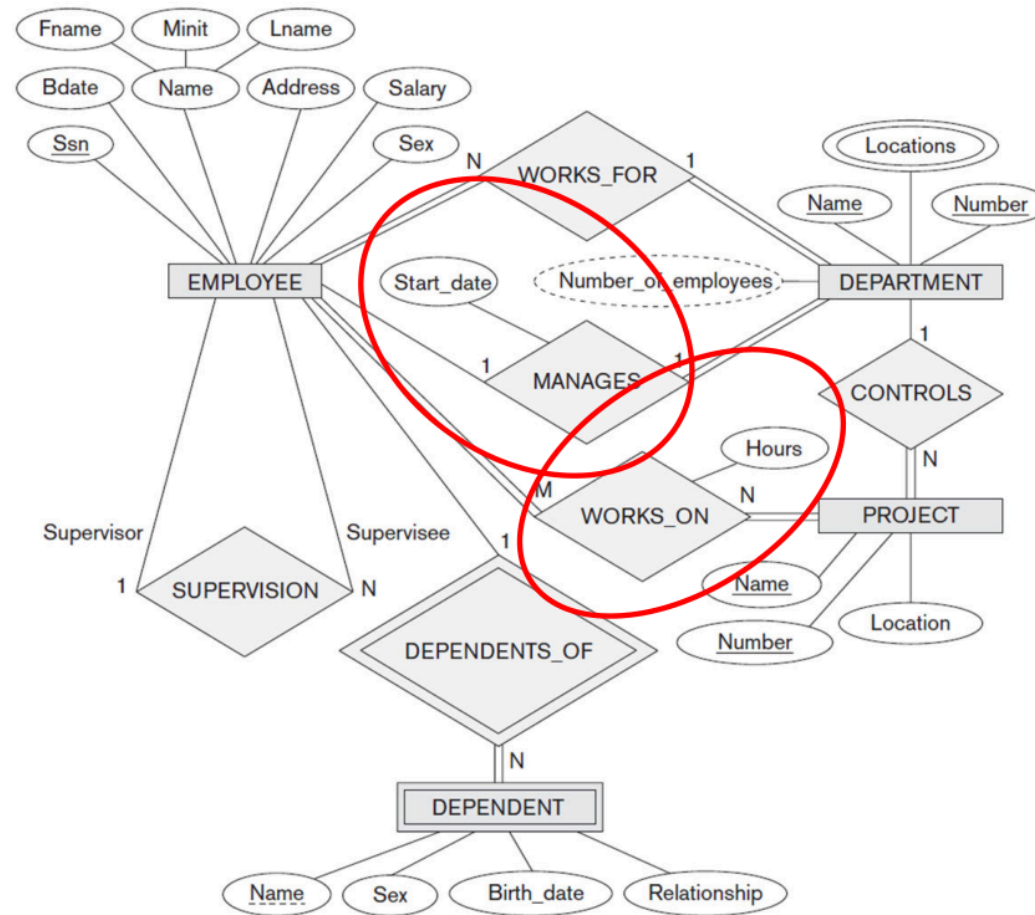
Company Example



Attributes

- Relationship types can also have attributes
- Notice that attributes of 1:1 or 1:N relationship types can be migrated to one of the participating entity types
- For M:N relationship types, some attributes may be determined by the combination of participating entities in a relationship instance, not by any single entity Such attributes must be specified as relationship attributes

Company Example



How to define them?

- Relationship between entity types
- Good naming
- More than one relationship?
 - ▶ Maybe different meanings, roles
 - ▶ Example for role: Supervisor, Supervisee
- Cardinalities
- Mandatory/optional
- Attributes for Relationship Type?

Constraints

- Three categories
 1. Constraints that are inherent in the data model **inherent model-based constraints** or **implicit constraints** Example: no duplicate tuples in a relation
 2. Constraints that can be directly expressed in schemas of the data model **schema-based constraints** or **explicit constraints** Example: Domain constraints, primary key (entity integrity constraints), constraints on NULL, and **referential integrity constraints**
 3. Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs **application-based** or **semantic constraints** or **business rules**

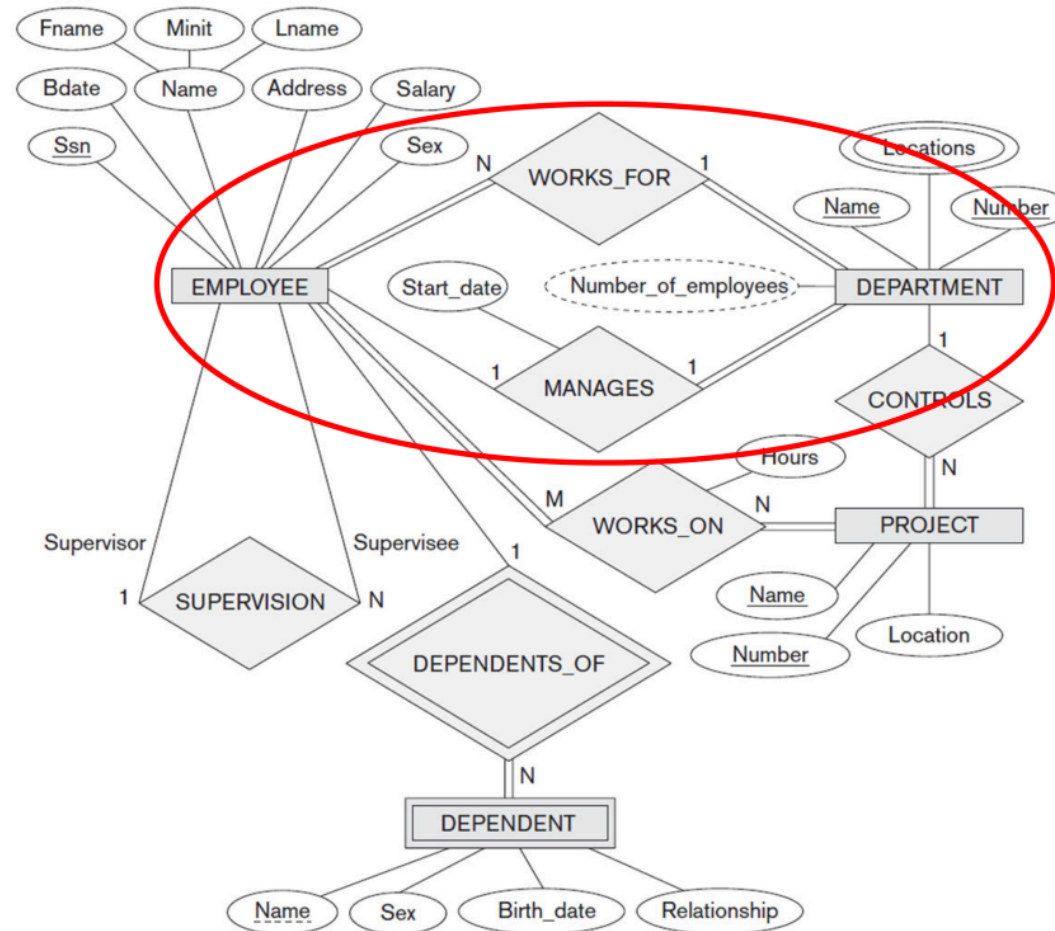
Referential Integrity Constraint

- It is defined between two relations
- It is used to maintain the consistency among tuples in the two relations: a tuple in one relation that refers to another relation must refer to an existing tuple in that relation
- **Foreign key**: a set of attributes FK in relation schema R_1 is a foreign key of R_1 that references relation R_2 if it satisfies the following rules:
 - ▶ 1. The attributes in FK have the same domain(s) as the primary key attributes PK of R_2 ; the attributes FK are said to **reference** or **refer to** the relation R_2
 - ▶ 2. A value of FK in a tuple t_1 of the current state $r_1(R_1)$ either occurs as a value of PK for some tuple t_2 in the current state $r_2(R_2)$ or is NULL. In the former case, we have $t_1[\text{FK}] = t_2[\text{FK}]$, and we say that the tuple t_1 references or refers to the tuple t_2

Referential Integrity Constraints

- A foreign key can refer to its own relation
- Foreign keys are depicted with a directed arrow: The arrowhead may point to the primary key
- All integrity constraints can be defined with the DDL, thus the DBMS can automatically enforce them

Company Example



Company Example

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

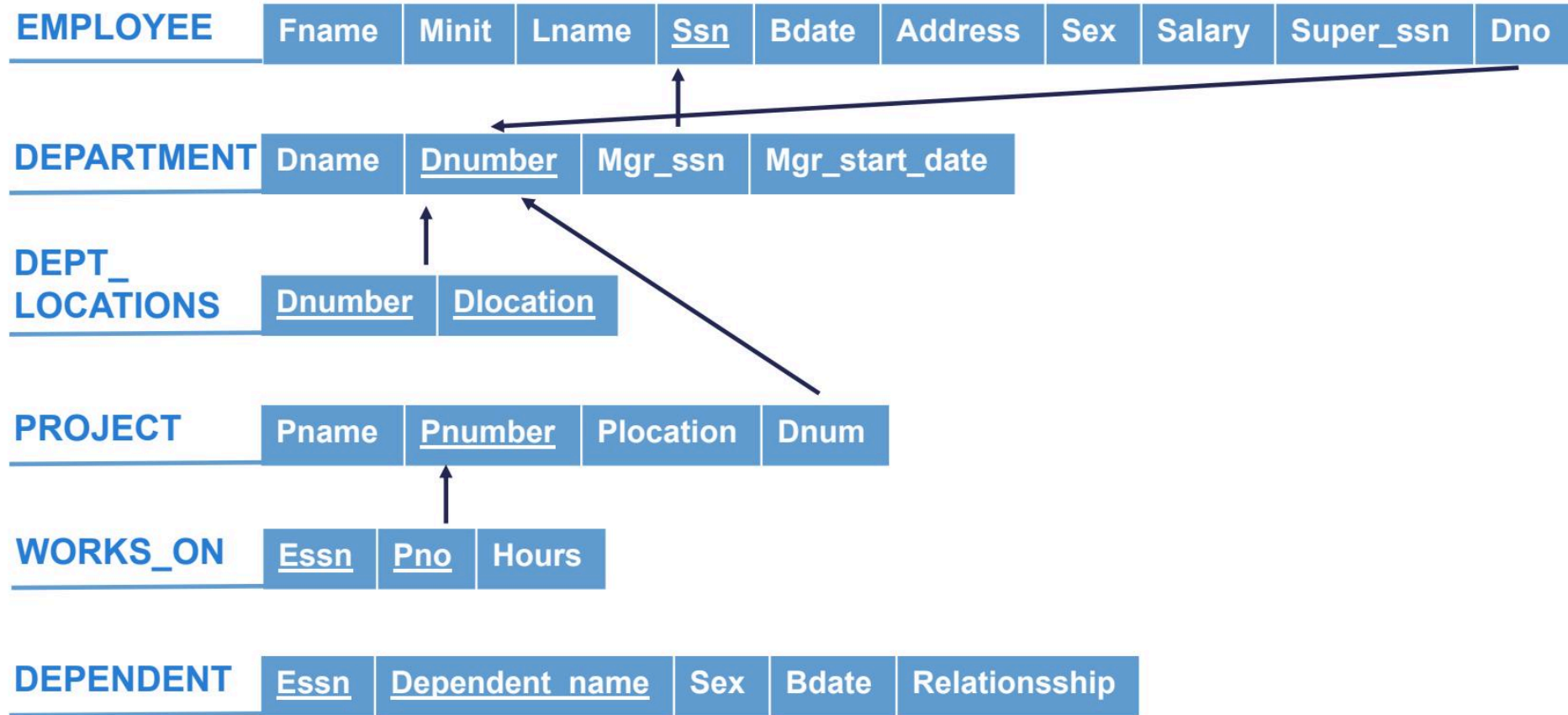
<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

Notation of Foreign Keys

- There are several ways for the notation of relational schemas, especially for representing foreign keys
 - ▶ Option 1:
 - Foreign Keys can be represented with arrows
 - This notation is used in the lecture slides and in the book “Fundamentals of Database Systems” from Elmasri and Navathe
 - Advantage: Each FK-arrow connects the referencing attribute and referenced attribute, so the involved relations are obvious
 - Option 2:
 - Foreign Keys can be represented with addition (FK) within the referencing attribute
 - This notation is used in the laboratory of Mr. Ocker
 - Advantage: This notation is more readable for large, complex schemas

- Both notations are correct and may be used within the examination

Notation of Foreign Keys



Notation of Foreign Keys

<u>EMPLOYEE</u>	Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-----------------	-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

<u>DEPARTMENT</u>	Dname	<u>Dnumber</u>	Mgr_ssn (FK)	Mgr_start_date
-------------------	-------	----------------	--------------	----------------

<u>DEPT_</u> <u>LOCATIONS</u>	<u>Dnumber (FK)</u>	<u>Dlocation</u>
----------------------------------	---------------------	------------------

<u>PROJECT</u>	Pname	<u>Pnumber</u>	Plocation	Dnum (FK)
----------------	-------	----------------	-----------	-----------

<u>WORKS_ON</u>	<u>Essn</u>	<u>Pno (FK)</u>	Hours
-----------------	-------------	-----------------	-------

<u>DEPENDENT</u>	<u>Essn</u>	<u>Dependent name</u>	Sex	Bdate	Relationship
------------------	-------------	-----------------------	-----	-------	--------------

Mapping of ERM

- 1. Mapping of regular entity types
- 2. Mapping of weak entity types
- **3. Mapping of binary 1:1 relationships**
- **4. Mapping of binary 1:n relationships**
- **5. Mapping of binary m:n relationships**
- 6. Mapping of multivalued attributes
- 7. Mapping of n-ary relationships

Mapping of Binary 1:1

- For each binary 1:1 relationship type R in the ER schema, identify the relations S and T that correspond to the entity types participating in R
- There are three possible approaches:
 - ▶ 1. The foreign key approach
 - ▶ 2. The merged relationship approach
 - ▶ 3. The cross-reference or relationship relation approach

Mapping of Binary 1:1

1. The foreign key approach

- Choose one of the relations S and include as a foreign key in S the primary key of T
- It is better to choose an entity type with total participation in R in the role of S
- Include all the simple attributes (or simple components of composite attributes) of the 1:1 relationship type R as attributes of S

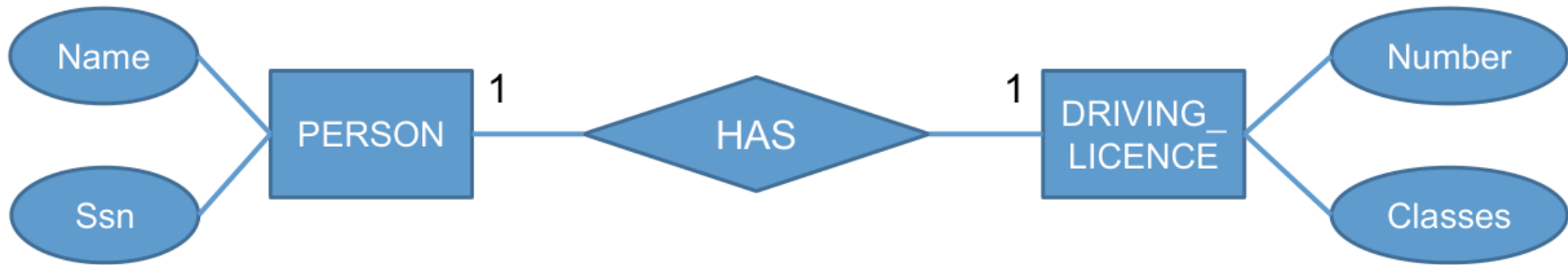
Mapping of Binary 1:1



- Mapping of relationship type MANAGES
 - ▶ DEPARTMENT serves as S
 - ▶ EMPLOYEE serves as T
- Attribute SSN is renamed in MGR_SSN in DEPARTMENT
- Attribute START_DATE is renamed in Mgr_start_date in DEPARTMENT
- It is also possible to include primary key of S as foreign key in T
- For the mapping, a **UNIQUE-Constraint** must be used!
 - ▶ Otherwise, an employee could manage several departments!

Mapping of Binary 1:1

- For the mapping, a UNIQUE-Constraint must be used!



Mapping of Binary 1:1

2. Merged relation approach

- Merge the two entity types and the relationship into a single relation
- This is possible when both participations are total, as this would indicate that the two tables will always have the exact same number of tuples

EMPLOYEE_								
<u>IN_DEPARTMENT</u>	Fname	Minit	Lname	<u>Ssn</u>	Dname	Dnumber	...	

Mapping of Binary 1:1

3. The cross-reference or relationship relation approach

- Set up a third relation R for the purpose of cross-referencing the primary keys of the two relations S and T representing the entity types
- This approach is required for binary M:N relationships
- The relation R will include the primary key attributes of S and T as foreign keys to S and T
- The primary key of R will be one of the two foreign keys, and the other foreign key will be a unique key of R

MANAGES

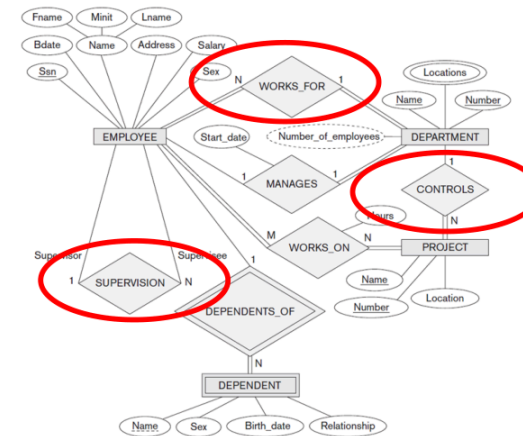
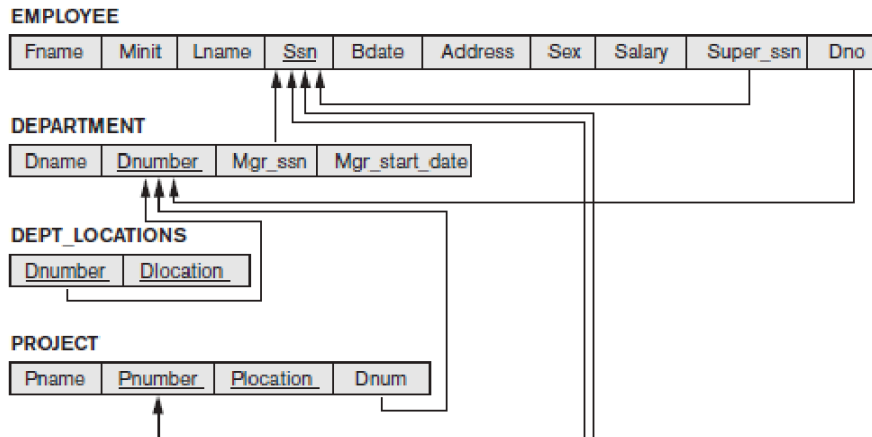
Ssn

Dnumber

Mapping of Binary 1:N

- For each regular binary 1:N relationship type R, identify the relation S that represents the participating entity type at the N-side of the relationship type
- Include as foreign key in S the primary key of the relation T that represents the other entity type participating in R
- Include any simple attributes (or simple components of composite attributes) of the 1:N relationship type as attributes of S
- An alternative approach: use the relationship relation and create a separate relation

Mapping of Binary 1:N



- Relationship type WORKS_FOR: Attribute Dno as foreign key in EMPLOYEE
- Relationship type SUPERVISION: Attribute Super_ssn as foreign key in EMPLOYEE
- Relationship type CONTROLS: Attribute Dnum as foreign key in PROJECT

Mapping of Binary 1:N - Total Participation

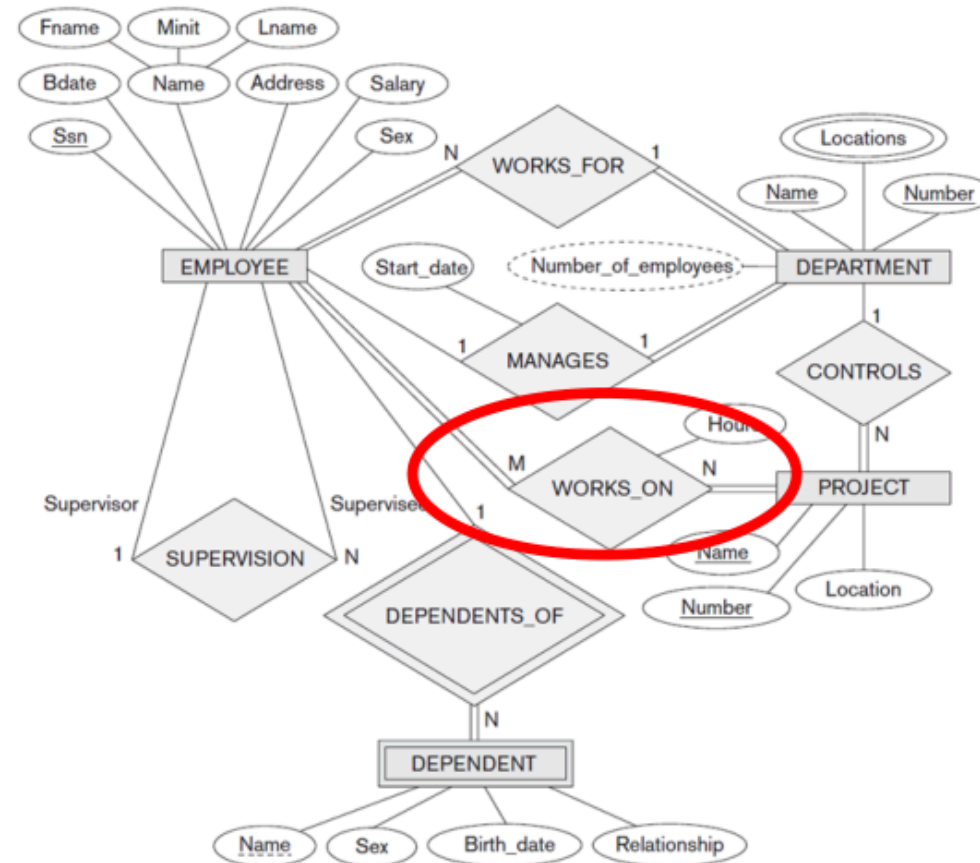
Total and Partial Participation should be mapped as well

- For participation definitions on the “1” side, a constraint assures the requirement
 - ▶ Total Participation 1:m → NOT NULL on FK
 - ▶ Partial Participation “0:m” → NULL on FK
- For participation definitions on the “m” side, there is a problem
 - ▶ These types (1:n vs. “1:0n”) are not distinguishable in Relational Model
 - ▶ These types of Total Participation cannot be implemented / enforced using SQL-DDL!

Mapping of Binary M:N

- For each binary M:N relationship type R, create a new relation S to represent R
- Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types; their combination will form the primary key of S
- Also include any simple attributes of the M:N relationship type (or simple components of composite attributes) as attributes of S
- Notice that a M:N relationship type can not be represented by a single foreign key attribute in one of the participating relations (as in 1:1 or 1:N relationship types) because of the M:N cardinality ratio

Mapping of Binary M:N



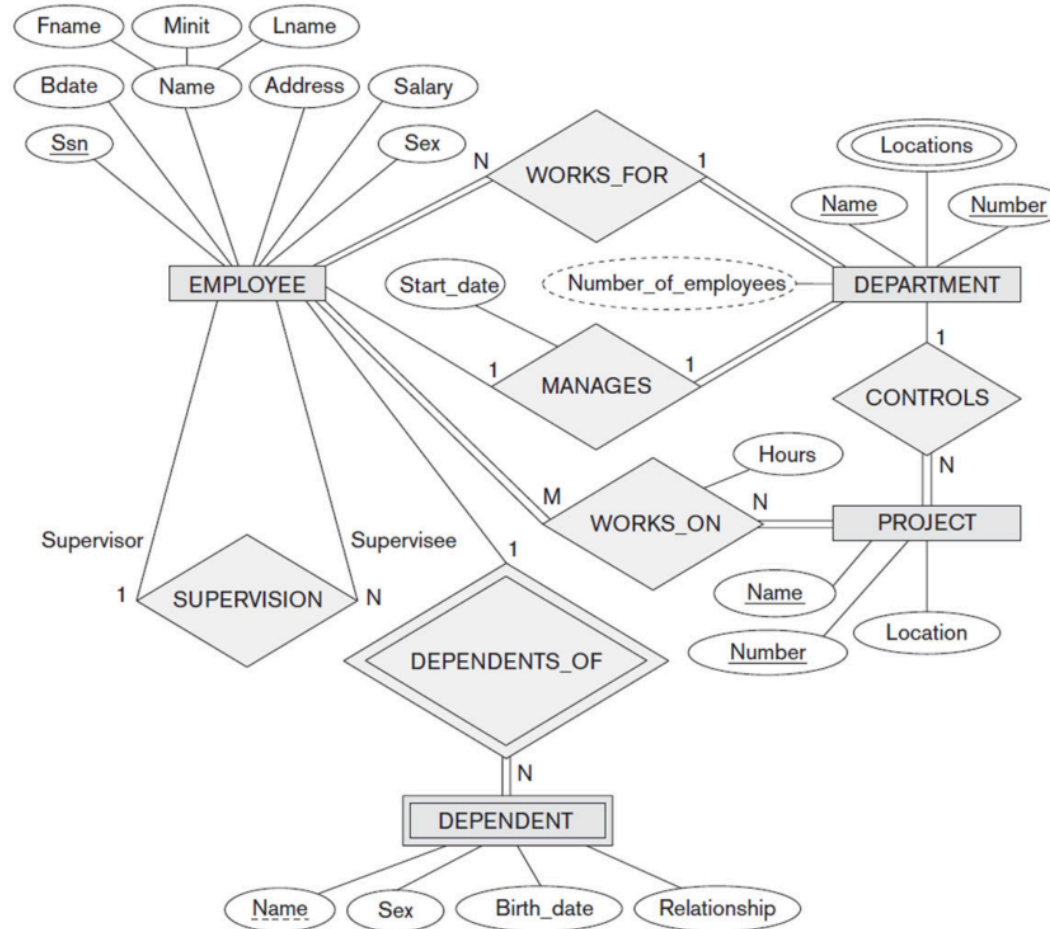
Mapping of Binary M:N

- Attribute Ssn is renamed in Essn in WORKS_ON
- Attribute Pname is renamed in Pno in DEPARTMENT
- Primary key is the combination {Essn, Pno}

i Info

The existence dependency between EMPLOYEE and PROJECT should be specified on the foreign keys in the relation corresponding to the relationship R (ON UPDATE and ON DELETE)

Example Company



Example Company

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

PROJECT

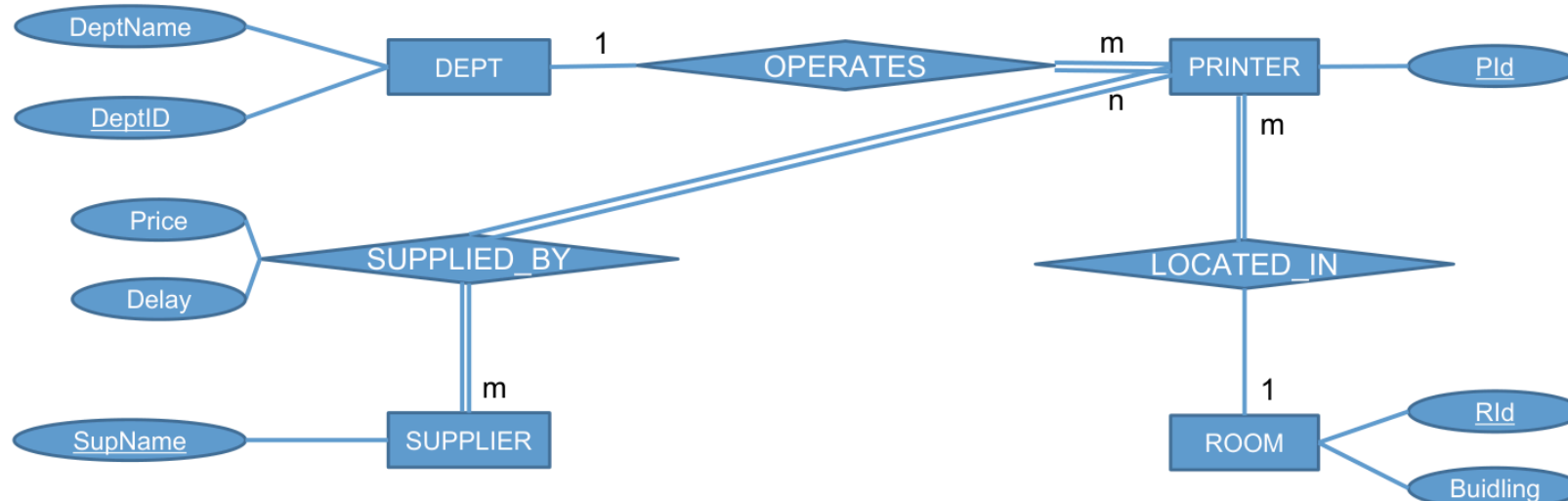
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

Assignment Office - Convert ERD to RM

Departments, identified by ID, operate a variety of printers, each located in a particular room in a particular building. Printers are supplied by a number of suppliers, identified by name, with each supplier charging a different price for a given printer, but also providing different delivery delays, measured in days. A given room can have any number of printers, including none.



Mapping of ERM to RM

1. Mapping of regular entity types ✓
2. Mapping of weak entity types
3. Mapping of binary 1:1 relationships ✓
4. Mapping of binary 1:n relationships ✓
5. Mapping of binary m:n relationships ✓
6. Mapping of multivalued attributes
7. Mapping of n-ary relationships

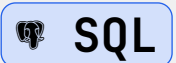
Create Table - Constraints

- Referential integrity is specified via the **FOREIGN KEY**
- FK relates two tables
- Referenced table must exist already
- Referenced column must be **UNIQUE**
 - ▶ Best to use PK
 - ▶ If not PK: need to specify (column)

Create Table - Constraints Syntax

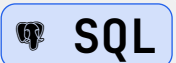
- As Column Constraint:
 - ▶ Only if the foreign key is one single attribute (and not combined)

```
1 [CONSTRAINT < constraintname > ]  
2     REFERENCES < tablename >[( column )] [< action >]
```



- As Table Constraint:

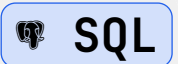
```
1 [CONSTRAINT < constraintname >]  
2     FOREIGN KEY (< column list >)  
3     REFERENCES < tablename >[(< column list >)]  
4 [< action >]
```



Create Table - Constraints Syntax

- Example column constraint:

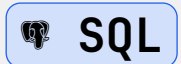
```
1 CREATE TABLE Department
2 ( Dname VARCHAR(15) NOT NULL,
3   Dnumber INT NOT NULL,
4   Mgr_ssn CHAR(9) REFERENCES Employee(Ssn) ,
5   Mgr_start_date DATE,
6   PRIMARY KEY (Dnumber) ,
7   UNIQUE (Dname));
```



Create Table - Constraints Syntax

- Example table constraint:

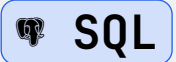
```
1 CREATE TABLE Department
2 ( Dname VARCHAR(15) NOT NULL,
3   Dnumber INT NOT NULL,
4   Mgr_ssn CHAR(9) NOT NULL,
5   Mgr_start_date DATE,
6   PRIMARY KEY ( Dnumber ),
7   UNIQUE ( Dname ),
8   FOREIGN KEY ( Mgr_ssn ) REFERENCES Employee ( Ssn ) );
```



Create Table - Constraints

- `<action>`:
 - ▶ How to react on changes to the referenced table
- The default action: reject the update operation (**RESTRICT** option)

```
1  action ::= ON {UPDATE | DELETE}
2  {NO ACTION | SET NULL | SET DEFAULT | CASCADE}
```

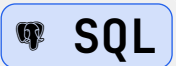


Create Table - Constraints

- Options:
 - ▶ **SET NULL** Value of foreign key is set to NULL
 - ▶ **SET DEFAULT** Value of foreign key is set to a default value
 - ▶ **CASCADE** Value of foreign key is updated
- For example:
 - ▶ **ON DELETE CASCADE** Delete all referencing tuples
 - ▶ **ON UPDATE CASCADE** Change Value of the foreign key attribute(s)
- General Rule for using **CASCADE**:
 - ▶ For “relationship” relations
 - ▶ For multivalued attributes
 - ▶ For relations that represent weak entity types

Create Table - Constraints

```
1  CREATE TABLE Employee
2  ( . . . ,
3  Dno INT NOT NULL DEFAULT 1,
4  CONSTRAINT EMPPK PRIMARY KEY (Ssn),
5  CONSTRAINT EMPSUPERFK FOREIGN KEY (Super_ssn) REFERENCES
   Employee(Ssn)
6  ON DELETE SET NULL ON UPDATE CASCADE,
7  CONSTRAINT EMPDEPTFK FOREIGN KEY(Dno) REFERENCES
   Department(Dnumber)
8  ON DELETE SET DEFAULT ON UPDATE CASCADE
9  );
```



ALTER TABLE

- For modifying an existing relation
 - ▶ COLUMN: **ADD, DROP, MODIFY**
 - ▶ CONSTRAINT: **ADD, DROP**
 - ▶ TABLE: **RENAME**

ALTER TABLE - Column

Syntax for altering a table:

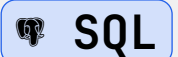
```
1 ALTER TABLE < relationname > . . .
```



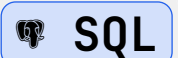
```
1 ADD [ COLUMN ] < column > < type >  
2      [ < col\_constraint > [ . . . ]
```



```
1 DROP [COLUMN] <column> [RESTRICT | CASCADE]
```



```
1 RENAME COLUMN <column> TO <new_column>
```



ALTER TABLE - Column

- Syntax for altering a table:

```
1 ALTER TABLE < relationname > . . .
```



Modification of columns vendor-specific: Oracle:

```
1 ... MODIFY < column > < type > [< col\_constraints > [...]]
```



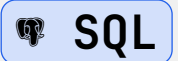
MySQL:

```
1 ... CHANGE [ COLUMN ] < column > < type > ...
```



ALTER TABLE - Column

```
1 ALTER TABLE COMPANY.EMPLOYEE ADD COLUMN Job TEXT(12);
```



- Inserting values for the new column:
 - ▶ Default is NULL → NOT NULL constraint is not allowed
 - ▶ Using default clause
 - ▶ Using UPDATE individually on each tuple

ALTER TABLE - Constraints

Syntax for adding a new constraint:

```
1 ALTER table ADD < tableconstraint > ;
```



- Add a foreign key (instead of within create table statement):

```
1 ALTER TABLE DEPARTMENT
```



```
2 ADD CONSTRAINT DEPTMGRFK FOREIGN KEY (Mgr_ssn) REFERENCES  
  EMPLOYEE(Ssn)
```

```
3 ON DELETE RESTRICT ON UPDATE CASCADE;
```

ALTER TABLE - Constraints

Syntax for dropping an existing constraint:

```
1 ALTER TABLE < tablename > < alterstatement >
```



```
1 < alterstatement > ::=
```



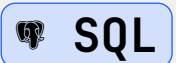
```
2 DROP PRIMARY KEY |
```

```
3 DROP FOREIGN KEY < keyname > |
```


ALTER TABLE - Rename

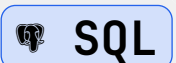
- Syntax for renaming an existing table:
- Oracle, MySQL:

```
1 RENAME TABLE < relationname > TO < newrelationname >
```



- PostgreSQL, MySQL:

```
1 ALTER TABLE < name > RENAME TO < new_name >
```



Homework

- Company Example
 - ▶ Implement all relationship types from the ERM in your database
 - ▶ Think also about the cardinalities and participation constraints of these relationship types
 - ▶ What should be the behavior of these relations if data changes?
 - ▶ Try SQL statements for inserting, updating, and deleting data
- Implement the printer example in your database
- Think about your own, individual example (e.g., contact list)
 - ▶ Implement all relationship types from the ERM in your database
 - ▶ Think also about the cardinalities and participation constraints
 - ▶ What should be the behavior of these relations if data changes?
 - ▶ Try SQL statements for inserting, updating, and deleting data

3. License Notice

3.1 Attribution

The basis of this course stems from: Professor Dr. Ulrike Herster The following license applies to all unmodified and modified material used throughout this course.

! License

The publication or sharing of slides, image and sound recordings of this course are not permitted.

© Professor Dr. Ulrike Herster

This video and the presented script are protected by copyright. The use is only permitted in relation with the course of study. It is not permitted to forward or republish it in other places (especially in the internet).